Лекция 3. Перегрузка операций. Преобразование типов

Перегрузка операций

Перегрузка операций – это одна из самых мощных и полезных возможностей ООП. Её применение позволяет существенно упростить написание программ, сделать их понятнее за счет применения интуитивно понятных обозначений, таких как +, *, [ ] и т.д.

Перегрузка операций, по сути, дает возможность переопределить стандартный язык программирования. Используя классы для создания новых типов переменных и перегрузку для определения операций над этими типами, можно, по сути, создать новый, собственный язык программирования, заточенный под конкретную задачу, и, таким образом, перейти от общего к предметно-ориентированному программированию.

Благодаря перегрузке операций любой класс можно сделать таким, что он будет вести себя подобно встроенному типу данных. В классе можно перегрузить любые из следующих операций:

+	-	*	/	%	^	&	|	~	!	=
<	>	+=	-=	*=	/=	%=	^=	&=	|=	<<
>>	>>=	<<=	==	!=	<=	>=	&&	||	++	-
( )			[ ]	new	delete	new[ ]	delete [ ]			

Нельзя перегружать операции

. -> .* :: ?:

Функции-операции, реализующие перегрузку операций, имеют вид

operator операция ([операнды]) ;

Если функция-операция является элементом класса, то первый операнд соответствующей операции будет самим объектом, для которого вызвана функция-операция. В случае одноместной операции список параметров будет пуст. Для двухместных операций функция

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

будет иметь один параметр, соответствующий второму операнду. Если функция-операция не является элементом класса, она будет иметь один параметр в случае одноместной операции и два - в случае двухместной.

Для перегрузки операций существуют такие правила:

- Приоритет и правила ассоциации для перегруженных операций остаются теми же самыми, что и для операций над встроенными типами.
- Нельзя изменить поведение операции по отношению к встроенному типу.
- Функция-операция должна быть либо элементом класса, либо иметь один или несколько параметров типа класса.
- Функция-операция не может иметь аргументов по умолчанию.

В качестве примера приведем перегрузку операции сложения и умножения (как скалярного произведения) для класса, описывающего вектор в пространстве с координатами x, y, z.

```
class my_vector        // Объявляем класс - вектор длины 3
{
public:                // с элементами-данными x, y и z - компонентами вектора
   double x;
   double y;
   double z;
   double operator*(my_vector v2);      //Переопределяем оператор умножения
   my_vector operator+(my_vector v2);      //Переопределяем оператор сложения
};

double my_vector::operator*(my_vector v2)
{
   double help;
   help=x*v2.x + y*v2.y + z*v2.z;      //переопределяем оператор умножения * как
   return help;                        //скалярное произведение векторов
};
```

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

```
my_vector my_vector::operator+(my_vector v2)
{
   my_vector help;
   help.x = x + v2.x;
   help.y = y + v2.y;
   help.z = z + v2.z;              //переопределяем оператор сложения + как
   return help;                              //сумму векторов
};
```

После этого в программе для объектов класса *my_vector* будут доступны операции сложения и умножения:

```
my_vector v1, v2, v3;
double a;

v1 = v2 + v3;
a = v1 * v2;
```

При перегрузке операций необходимо помнить следующее:

- C++ не умеет образовывать из простых операций более сложные. Например, в классе со сложением строк мы определили присваивание и сложение; но это не значит, что тем самым будет автоматически определено присвоение суммы (+=). Такую операцию нужно реализовывать отдельно.
- Невозможно изменить синтаксис перегруженных операций. Одноместные операции должны быть одноместными, а двухместные - двухместными.
- Нельзя изобретать новые обозначения операций. Возможные операции ограничиваются тем списком, что приведен в начале этого раздела.
- Желательно сохранять смысл перегружаемой операции. Например, конкатенация - естественная семантика сложения для строк.

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

Операции не обязательно объявлять членами класса. Скажем, предыдущий пример с перегрузкой операций сложения и умножения для класса *my_vector* можно реализовать и так:

```
class my_vector
{           // Объявляем класс - вектор длины 3
public:                 // с элементами-данными x, y и z - компонентами вектора
   double x;
   double y;
   double z;
};

double operator*( my_vector v1, my_vector v2)
{
   double help;
   help=v1.x*v2.x + v1.y*v2.y +v1. z*v2.z;     //переопределяем оператор умножения * как
   return help;                              //скалярное произведение векторов
};

my_vector operator+( my_vector v1, my_vector v2)
{
   my_vector help;
   help.x = v1.x + v2.x;
   help.y = v1.y + v2.y;
   help.z = v1.z + v2.z;      //переопределяем оператор сложения + как
   return help;                              //сумму векторов
};
```

Несколько слов о перегрузке унарных операций. Имеется особенность синтаксиса при перегрузке операций с префиксной и постфиксной формой ++ (инкремент) и -- (декремент). В случае перегрузки префиксной формы используют следующий синтаксис переопределения:

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

void operator++( );

Если требуется переопределить постфиксную форму, то прототип перегружаемой операции будет таким:

void operator++(int);

Различие состоит лишь в том, что у постфиксной формы в скобках стоит *int*. Здесь *int* не играет роль аргумента и не означает целое число. Это просто сигнал для компилятора, чтобы использовалась постфиксная версия оператора.

Имеется также особенность перегрузки операции [ ] – индексация массива. Так как данная операция часто используется не только для доступа на чтение, но и для доступа на запись (то есть что-то типа *x[i] = y*), то целесообразно сделать так, чтобы перегруженная функция *operator []* возвращала свое значение по ссылке. Это будет выглядеть примерно следующим образом:

int& operator[](int i);

Преобразование типов

Объекты класса могут быть преобразованы в другие типы данных с помощью операций приведения типа или конструкторов преобразования.

Конструктором преобразования называется функция-конструктор, имеющая в качестве параметра объект какого-либо класса. Если конструктор класса A имеет единственный параметр типа B, то объект класса B может быть неявно преобразован в объект класса A с помощью такого конструктора, то есть компилятор может сам вызвать такой конструктор и из B сделать A. Это происходит в следующем примере:

class Hold

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

```
{
private:
   char *str;
public:
   Hold(const char*);
   //...
};

main ( )
{
   Hold mainObj = "This is a local object in main.";
   //. . .
   return 0;
}
```

Здесь объявленный в классе конструктор *Hold(const char*);* является, по сути, конструктором преобразования. Если вызовы конструктора с одним параметром в качестве конструктора преобразования необходимо запретить, то его нужно объявить с ключевым словом *explicit*. Пример:

```
class Hold
{
private:
   char *str;
public:
   explicit Hold(const char*);
   //. . .
};

main ( )
{
   //Теперь неявное преобразование недопустимо:
```

```
    // Hold mainObj = "This is a local object in main.";
    //Но можно вызвать конструктор с параметром явным образом:
    Hold mainObj("This is a local object in main.");
    return 0;
}
```

В классе можно определять элементы-функции, которые будут обеспечивать явное или неявное преобразование объектов данного класса в объекты других классов. Эти функции называют операциями приведения типа или процедурами преобразования. Они имеют следующий синтаксис:

```
operator имя_нового_типа( );
```

Процедуры преобразования характеризуются следующими правилами:

- У процедуры преобразования нет параметров;
- Для процедуры преобразования не специфицируется явно тип возвращаемого значения. Подразумевается тип, имя которого следует за ключевым словом *operator*.

В следующем примере процедура преобразования преобразует время, выраженное в часах, минутах и секундах в соответствующее число секунд.

```
class Time
{
public:
    int hr, min, sec; public:
    Time(int h, int m, int s): hr(h), min(m), sec(s) {}  // Конструктор.
    operator int( ); // Процедура преобразования времени в целое число.
};

Time::operator int( )
{
    return (3600*hr + 60*min + sec); // Преобразует время в число секунд от начала суток
```

```
}

main( )
{
   int h = 7;
   int m = 40;
   int s = 10;
   int d;

   Time t (h, m, s);

   d = t; // Здесь будет неявно вызвана функция operator int( );
}
```

Таким образом, преобразование от встроенного типа к определенному пользователем может быть выполнено только при создании объекта с помощью конструктора преобразования. А преобразование от типа, определенного пользователем, к встроенному типу выполняется с помощью определенных в классе операций преобразования типа.

Преобразование типов может быть также выполнено с помощью переопределения операции присваивания.

Вопросы к лекции 3

1. Для чего в C++ применяется перегрузка операций

2. Истинно ли следующее утверждение: операция >= может быть перегружена?

3. Сколько аргументов требуется для определения перегруженной унарной операции?

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

4. Сколько аргументов требуется для определения перегруженной бинарной операции?

5. Чем отличается действие операции ++ в префиксной форме от её действия в постфиксной форме?

6. Истинно ли следующее утверждение: перегруженная операция всегда требует на один аргумент меньше, чем количество операндов?

7. Когда перегружается операция арифметического присваивания, то результат
   a. Передается объекту справа от операции
   b. Передается объекту слева от операции
   c. Передается объекту, вызвавшему операцию
   d. Должен быть возвращен

8. Какой механизм преобразования от определенного пользователем класса к встроенному типу может быть использован в языке C++?

9. Какой механизм преобразования от встроенного типа данных к определенному пользователем может быть использован в языке C++?

10. Истинно ли следующее утверждение: компилятор не выдаст сообщение об ошибке, если вы перегрузите операцию * для выполнения деления?

11. Если объект objA принадлежит классу A, объект objB принадлежит классу B, и требуется записать objA = objB, поместив при этом функцию преобразования в класс A, то какую разновидность процедуры преобразования типа можно использовать?

12. Существуют ли операции, которые нельзя перегружать?

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС

13. Что такое конструктор преобразования?

14. Для чего используется ключевое слово *explicit*

© В.М. Гриняк, профессор. каф. ИТС ВГУЭС